

Tailor Platform Web

Base de reference PHP, HTML, CSS et Bootstrap

Notes personnelles

September 12, 2026

Contents

1	Introduction	2
2	Etat du projet : Tailor Platform Web	2
2.1	Objectif	2
2.2	Design suivi	2
2.3	Structure du projet	2
2.4	Parcours V1 (identique au mobile)	3
2.5	Etat d'avancement	3
3	PHP : vue d'ensemble des notions	3
3.1	Classes en PHP : Realisation et Tailor	4
3.2	Namespace et autoload	6
3.3	Superglobales : \$_GET, \$_POST, \$_SESSION, \$_SERVER	7
3.4	Sessions PHP	8
3.5	header() et redirection	8
3.6	Traitement d'un formulaire	9
4	HTML, CSS et Bootstrap : vue d'ensemble	10
4.1	Grille Bootstrap : container, row, col	10
4.2	Composant Card	11
4.3	Formulaires Bootstrap	11
5	Dictionnaire des fiches PHP	12
5.1	Panier (classe)	12
5.2	Repository (CatalogueRepository, TailorRepository)	13
6	Cheat sheet rapide	13
6.1	Reference ultra rapide	13
6.2	Quel outil utiliser	13
7	Regles d'evolution de ce document	14

1 Introduction

Ce document est le pendant web de ma base de reference Dart/Flutter. Il suit la meme logique : il grandit avec le projet, il distingue ce qui est "deja appris", "a consolider", "nouveau" ou "avance", et il ne repart jamais de zero lors d'une mise a jour.

Le support principal utilise pour la partie orientee objet de PHP est le tutoriel *Programmez en oriente objet en PHP* (Victor Thuillier, OpenClassrooms). La syntaxe presentee ici est cependant celle de PHP moderne (8.x), avec les types stricts, ce qui differe legerement du tutoriel original plus ancien mais respecte les memes principes de base.

Beaucoup de notions de programmation ont deja ete vues cote Dart. Quand c'est le cas, ce document fait le lien plutot que de tout ré-expliciter depuis zero.

2 Etat du projet : Tailor Platform Web

2.1 Objectif

La version web reproduit exactement le meme perimetre que la V1 mobile Flutter :

Le tailleur vend des modeles et le client en achete.

Meme choix qu'en mobile pour cette V1 :

- pas de base de donnees ;
- pas de backend externe (PHP est lui-meme le backend, mais sans SGBD) ;
- pas d'authentification reelle ;
- pas de paiement reel ;
- pas de livraison reelle.

A la place d'une base de donnees, les donnees vivent dans la session PHP (`$_SESSION`). C'est l'equivalent web des `List` en memoire cote Flutter : les donnees disparaissent si on ferme completement le navigateur (la session expire).

2.2 Design suivi

Le site reprend la palette et l'esprit visuel d'une maquette fournie (couleurs terracotta, foret, sable, creme, typographie serif pour les titres). L'implementation reste volontairement simple : HTML, CSS et Bootstrap, sans framework JavaScript.

2.3 Structure du projet

```
tailor_platform_web/  
  src/  
    Model/  
      Tailor.php  
      Realisation.php  
    Support/  
      Panier.php  
      CartService.php  
    Data/  
      CatalogueRepository.php  
      TailorRepository.php  
  includes/
```

```

bootstrap.php
header.php
footer.php
public/
  index.php
  tailor_profile.php
  add_article.php
  catalogue.php
  article_detail.php
  cart.php
  order_confirmation.php
  assets/
    css/style.css
    vendor/bootstrap/css/bootstrap.min.css
    vendor/bootstrap/js/bootstrap.bundle.min.js

```

`public/` est la racine web (document root). `src/` et `includes/` contiennent du code PHP qui n'est jamais accédé directement par le navigateur : il est chargé par les pages de `public/`.

2.4 Parcours V1 (identique au mobile)

```

TAILLEUR
profil/catalogue
|
ajouter un modele
|
modele disponible dans le catalogue
|
CLIENT
catalogue
|
detail du modele
|
ajouter au panier
|
panier
|
calcul du total
|
commander
|
confirmation de commande
|
retour au catalogue

```

2.5 Etat d'avancement

L'ensemble du parcours ci-dessus est fonctionnel dans la première version du site web. Contrairement à la version Flutter (développée étape par étape), la version web a été construite d'un bloc en reprenant les règles déjà établies côté mobile.

3 PHP : vue d'ensemble des notions

Notion	Statut	Remarque
--------	--------	----------

Variables (\$), types	Nouveau	Similaire a Dart, mais prefixe \$ et typage plus souple
echo, affichage	Nouveau	
Tableaux indexes et associatifs	Nouveau	Jouent le role des List et Map de Dart
foreach	Nouveau	Equivalent du for-in de Dart
Conditions, operateurs	Deja appris	Meme logique qu'en Dart
Fonctions	Deja appris	Meme logique qu'en Dart, syntaxe differente
Classes, objets, \$this	Nouveau	Bases deja connues via Dart, syntaxe PHP a apprendre
Visibilite (private, public, protected)	Nouveau	Rapproche du principe d'encapsulation
Constructeur __construct	Nouveau	Equivalent du constructeur Dart
Namespaces	Nouveau	Organise les classes, evite les conflits de noms
Autoload (spl_autoload_register)	Nouveau	Charge les classes automatiquement
Superglobales (\$_GET, \$_POST, \$_SESSION, \$_SERVER)	Nouveau	Coeur de la communication entre pages en PHP
Sessions PHP	Nouveau	Remplace la base de donnees pour cette V1
include / require	Nouveau	Assemble les pages a partir de morceaux communs (header, footer)
header() et redirection	Nouveau	Equivalent web de Navigator.push/pop
Traitement de formulaire (POST)	Nouveau	
Validation de donnees utilisateur	Nouveau	
Exceptions (try/catch)	Avance	Pas encore utilise dans le projet
Interfaces, heritage, abstract	Avance	Vus dans le tutoriel PHP, pas encore utilises dans le projet

3.1 Classes en PHP : Realisation et Tailor

Statut : Nouveau

C'est quoi. Comme en Dart, une classe decrit un modele de donnee et un objet en est une instance. La grande difference de syntaxe : les attributs sont precedes de \$ et on accede a un attribut ou une methode avec -> au lieu de ..

Comparaison rapide avec Dart.

Concept	Dart	PHP
Definir une classe	<code>class Realisation { }</code>	<code>class Realisation { }</code>

Attribut	String title;	private string \$title;
Constructeur	Realisation({required this, title})	public function __construct(string \$title)
Acceder a un attribut/methode	objet.title	\$objet->getTitle()
Soi-meme	this	\$this

Syntaxe minimale.

```
<?php

declare(strict_types=1);

namespace App\Model;

class Realisation
{
    private int $id;
    private string $title;
    private string $description;
    private float $price;

    public function __construct(int $id, string $title, string $description, float $price)
    {
        $this->id = $id;
        $this->title = $title;
        $this->description = $description;
        $this->price = $price;
    }

    public function getId(): int
    {
        return $this->id;
    }

    public function getTitle(): string
    {
        return $this->title;
    }

    public function getDescription(): string
    {
        return $this->description;
    }

    public function getPrice(): float
    {
        return $this->price;
    }
}
```

Pourquoi des attributs private avec des getters. C'est le principe d'encapsulation expliquer dans le tutoriel PHP OOP : personne en dehors de la classe ne doit pouvoir modifier directement \$title ou \$price. On passe obligatoirement par une methode publique (ici un getter) pour lire la valeur. Cela protege l'objet contre des modifications incoherentes.

Créer un objet.

```
$realisation = new Realisation(0, 'Costume traditionnel', 'Costume deux pieces sur
    mesure', 45000.0);

echo $realisation->getTitle();
```

Erreurs frequentes.

- Oublier `$this->` dans le constructeur : sans lui, la variable reste locale a la methode et l'attribut de l'objet n'est jamais rempli.
- Declarer un attribut `public` par facilite, ce qui casse l'encapsulation.
- Oublier le point-virgule a la fin d'une instruction (erreur frequente venant de Dart ou l'oubli est parfois moins visible).

Dans Tailor Platform. `App\Model\Realisation` et `App\Model\Tailor` sont la copie conforme, en PHP, des classes Dart du meme nom. Elles n'ont aucune logique de session ou d'affichage : elles ne font que représenter une donnée.

3.2 Namespace et autoload

Statut : Nouveau

C'est quoi. Un namespace est un peu comme un dossier logique pour les classes. Il évite qu'une classe `Realisation` du projet entre en conflit avec une classe `Realisation` d'une bibliothèque externe.

Syntaxe minimale.

```
namespace App\Model;

class Realisation
{
    // ...
}
```

Pour utiliser la classe ailleurs :

```
use App\Model\Realisation;

$realisation = new Realisation(0, 'Titre', 'Description', 1000.0);
```

L'autoload. Sans autoload, il faudrait écrire un `require` pour chaque fichier de classe avant de l'utiliser. L'autoload automatise ce chargement : dès qu'une classe inconnue est utilisée, PHP appelle une fonction qui va chercher le bon fichier.

```
spl_autoload_register(static function (string $class): void {
    $prefix = 'App\\';

    if (!str_starts_with($class, $prefix)) {
        return;
    }

    $relativeClass = substr($class, strlen($prefix));
    $file = __DIR__ . '/../src/' . str_replace('\\', '/', $relativeClass) . '.php';
```

```

    if (is_file($file)) {
        require_once $file;
    }
});

```

Point important rencontre dans le projet. L'autoload doit être enregistré **avant** `session_start()`. En effet, PHP recharge automatiquement les objets stockés en session dès le démarrage de la session. Si la classe correspondante n'est pas encore connue à ce moment (autoload pas encore enregistré), l'objet récupéré est incomplet et inutilisable. C'est une erreur réelle que nous avons rencontrée et corrigée dans `includes/bootstrap.php`.

Dans Tailor Platform. Toutes les classes du projet vivent sous le namespace `App\` (par exemple `App\Model\Realisation`, `App\Data\CatalogueRepository`). L'autoload est configuré une seule fois, dans `includes/bootstrap.php`, et ce fichier est inclus en tout premier dans chaque page.

3.3 Superglobales : `$_GET`, `$_POST`, `$_SESSION`, `$_SERVER`

Statut : Nouveau

C'est quoi. Les superglobales sont des tableaux automatiquement remplis par PHP, et accessibles depuis n'importe quel fichier sans rien importer.

Superglobale	Role
<code>\$_GET</code>	Données envoyées dans l'URL (<code>?id=3</code>)
<code>\$_POST</code>	Données envoyées par un formulaire en méthode POST
<code>\$_SESSION</code>	Données qui persistent d'une page à l'autre pour un même visiteur
<code>\$_SERVER</code>	Informations sur la requête en cours (méthode HTTP, etc.)

Exemple : lire un identifiant dans l'URL.

```
$id = isset($_GET['id']) && is_numeric($_GET['id']) ? (int) $_GET['id'] : null;
```

Exemple : lire un champ de formulaire.

```
$title = trim((string) ($_POST['title'] ?? ''));
```

Exemple : connaître la méthode HTTP de la requête.

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // le formulaire a été soumis
}
```

Erreurs fréquentes.

- Lire `$_POST['title']` sans vérifier qu'il existe : erreur si le champ n'a pas été envoyé. On utilise `??` pour donner une valeur par défaut.
- Faire confiance aveuglément à une donnée venant de `$_GET` ou `$_POST` : il faut toujours la valider avant de l'utiliser.
- Oublier `(int)` ou `(float)` : ces données arrivent toujours sous forme de texte (`string`), même si elles ressemblent à des nombres.

Dans Tailor Platform. `article_detail.php` lit l'identifiant du modele via `$_GET['id']`. `add_article.php` lit le titre, la description et le prix via `$_POST`. `cart.php` declenche la commande en verifiant `$_SERVER['REQUEST_METHOD']`.

3.4 Sessions PHP

Statut : Nouveau

C'est quoi. Une session permet de conserver des donnees d'une page a l'autre pour un meme visiteur, alors que chaque requete HTTP est normalement independante des autres.

Comment la visualiser.

```
Visiteur A Visiteur B
$_SESSION (A) $_SESSION (B)
  cart: [Costume] cart: []
  realisations: [...] realisations: [...]
```

Chaque visiteur a sa propre session, identifiee par un cookie invisible envoye automatiquement par le navigateur.

Demarrer une session.

```
if (session_status() !== PHP_SESSION_ACTIVE) {
    session_start();
}
```

Lire et ecrire dans la session.

```
$_SESSION['cart'][] = $realisation; // ajouter
$item = $_SESSION['cart']; // lire
$_SESSION['cart'] = []; // vider
```

Ce que je dois retenir. `$_SESSION` se comporte comme un tableau normal, mais son contenu survit d'une page a l'autre. C'est ce qui remplace la base de donnees dans cette V1 : le "catalogue" et le "panier" sont juste des tableaux stockes en session.

Dans Tailor Platform. `CatalogueRepository` stocke la liste des `Realisation` dans `$_SESSION['realisations']`. `CartService` stocke le panier dans `$_SESSION['cart']`. Si le tailleur ajoute un modele, il est visible tant que la session dure, exactement comme le `cart.add()` en memoire cote Flutter.

3.5 header() et redirection

Statut : Nouveau

C'est quoi. `header('Location: ...')` dit au navigateur : va sur cette autre page. C'est l'equivalent web de `Navigator.push` ou `Navigator.pop` en Flutter, mais cote serveur.

Syntaxe minimale.

```
header('Location: tailor_profile.php');
exit;
```

`exit` est indispensable juste apres : sans lui, le reste du script continue de s'executer meme si le navigateur va deja changer de page.

Passer une donnée à la page suivante. Cote Flutter, on passe une donnée via le constructeur de la page. Cote web, l'équivalent le plus simple est de l'ajouter dans l'URL :

```
header('Location: order_confirmation.php?total=' . rawurlencode((string) $total));
exit;
```

Et on la relit sur la page d'arrivée avec `$_GET` :

```
$total = isset($_GET['total']) ? (float) $_GET['total'] : 0.0;
```

Erreurs fréquentes.

- Oublier `exit` après `header('Location: ...')`.
- Appeler `header()` après avoir déjà affiché du HTML : PHP refuse alors d'envoyer l'en-tête (erreur "headers already sent").
- Oublier `rawurlencode` sur une valeur qui pourrait contenir des caractères spéciaux dans l'URL.

Dans Tailor Platform. `add_article.php` redirige vers `tailor_profile.php` après un ajout réussi. `article_detail.php` redirige vers `catalogue.php` après l'ajout au panier. `cart.php` redirige vers `order_confirmation.php` en transmettant le total et le nombre d'articles dans l'URL.

3.6 Traitement d'un formulaire

Statut : Nouveau

Principe général. Une page qui contient un formulaire fait souvent deux choses à la fois : elle affiche le formulaire (méthode GET) et elle traite son envoi (méthode POST), dans le même fichier PHP.

Structure type utilisée dans le projet.

```
<?php

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // 1. récupérer les valeurs envoyées
    // 2. les valider
    // 3. si tout est correct, enregistrer puis rediriger
}

// 4. si on arrive ici, afficher le formulaire (première visite ou erreur)
?>

<form method="post">
    <input type="text" name="title">
    <button type="submit">Enregistrer</button>
</form>
```

Ce que je dois retenir. Le fait de re-afficher le formulaire (au lieu de rediriger) quand il y a une erreur permet de montrer les messages d'erreur au visiteur, avec ce qu'il avait déjà tapé.

Dans Tailor Platform. `add_article.php` suit exactement cette structure : vérification du titre, de la description et du prix, affichage des erreurs si besoin, sinon enregistrement via `CatalogueRepository::add()` puis redirection.

4 HTML, CSS et Bootstrap : vue d'ensemble

Notion	Statut	Remarque
Structure HTML de base	Nouveau	doctype, html, head, body
Formulaires HTML (form, input, textarea)	Nouveau	
CSS : selecteurs, proprietes	Nouveau	
CSS : variables (–ma-couleur)	Nouveau	Utilisees pour la palette du projet
Bootstrap : grille (container, row, col)	Nouveau	Equivalent visuel de Row/-Column/Expanded en Flutter
Bootstrap : composants (card, navbar, btn, form-control)	Nouveau	
JavaScript	Avance	Pas encore utilise dans le projet (Bootstrap est inclus pour plus tard)

4.1 Grille Bootstrap : container, row, col

Statut : Nouveau

C'est quoi. La grille Bootstrap permet d'organiser une page en lignes et colonnes, un peu comme Row et Column en Flutter, mais pensee pour s'adapter automatiquement a la largeur de l'ecran (responsive).

Comment la visualiser.

```
container
  row
    col col col
  [ A ] [ B ] [ C ]
```

Sur un ecran etroit (mobile), les colonnes peuvent se replacer les unes sous les autres automatiquement.

Syntaxe minimale.

```
<div class="container">
  <div class="row">
    <div class="col">Colonne 1</div>
    <div class="col">Colonne 2</div>
  </div>
</div>
```

Classes utiles rencontrees dans le projet.

Classe	Role
container	Centre le contenu et lui donne une largeur maximale
row	Cree une ligne de colonnes
col	Une colonne de largeur egale et automatique
row-cols-1 row-cols-sm-2 row-cols-md-3	Nombre de colonnes different selon la taille d'ecran
g-4	Espace (gap) entre les colonnes et les lignes
d-flex	Active flexbox sur l'element

<code>justify-content-between</code>	Espace les elements flexbox aux extremités
<code>align-items-center</code>	Centre verticalement les elements flexbox

Erreurs frequentes.

- Mettre une classe `row` directement sur le `body`, sans `container` autour.
- Mettre du contenu directement dans `row` sans passer par `col`.
- Oublier `g-4` (ou une classe de `gap`) et se retrouver avec des cartes collees les unes aux autres.

Dans Tailor Platform. Le catalogue client et le catalogue du tailleur utilisent `row row-cols-1 row-cols-sm` pour afficher les modeles en grille responsive, ce qui est l'équivalent visuel du `GridView.builder` cote Flutter.

4.2 Composant Card

Statut : Nouveau

C'est quoi. Une `card` Bootstrap est un bloc visuel avec un fond, une bordure et un espacement interieur, utilise pour presenter une information de maniere groupée (un modele, un produit, etc.).

Syntaxe minimale.

```
<div class="card">
  <div class="card-body">
    <h3 class="card-title">Titre</h3>
    <p class="card-text">Description</p>
  </div>
</div>
```

Dans Tailor Platform. Chaque modele du catalogue est affiche dans une `card`, avec le titre, la description, le prix et un bouton "Voir". C'est l'équivalent visuel du `Card` Flutter utilise dans le `GridView.builder` mobile.

4.3 Formulaires Bootstrap

Statut : Nouveau

Syntaxe minimale.

```
<div class="mb-3">
  <label for="title" class="form-label">Titre</label>
  <input type="text" class="form-control" id="title" name="title">
</div>
```

Classes utiles.

Classe	Role
<code>form-label</code>	Style du texte au-dessus du champ
<code>form-control</code>	Style standard d'un champ de saisie
<code>mb-3</code>	Marge en dessous, pour espacer les champs entre eux
<code>btn btn-primary</code>	Bouton principal (couleur d'accent du site)

btn btn-outline-secondary	Bouton secondaire, moins visible
alert alert-danger	Bloc de message d'erreur

Dans Tailor Platform. Le formulaire `add_article.php` utilise ces classes pour le titre, la description et le prix, avec un bloc `alert alert-danger` affiche en cas d'erreur de validation.

5 Dictionnaire des fiches PHP

5.1 Panier (classe)

Statut : Nouveau

C'est quoi. Une classe qui encapsule la logique du panier : ajouter un article, compter les articles, calculer le total, vider le panier.

Pourquoi une classe plutôt qu'un simple tableau. On pourrait se contenter d'un tableau de `Realisation` en session, comme cote Flutter (`List<Realisation> cart = []`). Mais en orientant objet, on regroupe les données (les articles) et les actions possibles sur ces données (`add`, `getTotal`, `clear`) dans une seule classe. Cela évite de recalculer le total à plusieurs endroits différents du code avec des risques d'erreur.

Syntaxe minimale.

```
class Panier
{
    private array $items;

    public function __construct(array $items = [])
    {
        $this->items = $items;
    }

    public function add(Realisation $realisation): void
    {
        $this->items[] = $realisation;
    }

    public function getTotal(): float
    {
        $total = 0.0;

        foreach ($this->items as $realisation) {
            $total += $realisation->getPrice();
        }

        return $total;
    }
}
```

Comparaison avec Dart. La méthode `getTotal()` ressemble volontairement à la fonction `calculerTotal()` écrite cote Flutter : même logique (une boucle qui accumule les prix), juste rangée à l'intérieur d'une classe plutôt que laissée comme fonction indépendante.

Dans Tailor Platform. `CartService` fait le lien entre cette classe `Panier` et la session PHP : il reconstruit un `Panier` a partir de `$_SESSION['cart']` a chaque page, pour pouvoir utiliser ses methodes.

5.2 Repository (CatalogueRepository, TailorRepository)

Statut : Nouveau

C'est quoi. Un repository est une classe dont le role est uniquement de fournir des donnees (les recuperer, les ajouter), sans se soucier de l'affichage. C'est une facon d'organiser le code pour que chaque classe ait une seule responsabilite claire.

Dans Tailor Platform. `CatalogueRepository` initialise le catalogue fictif si la session est vide, retrouve un modele par son identifiant, et ajoute un nouveau modele. `TailorRepository` fournit le tailleur unique de cette V1. Ces deux classes seront les premieres a changer le jour ou une vraie base de donnees sera ajoutee au projet : le reste du code (les pages) n'aura pas besoin de changer, puisqu'il ne fait qu'appeler leurs methodes.

6 Cheat sheet rapide

6.1 Reference ultra rapide

Element	Role en un mot
<code>\$_GET</code>	lire l'URL
<code>\$_POST</code>	lire un formulaire
<code>\$_SESSION</code>	memoriser entre les pages
<code>header('Location: ...')</code>	rediriger
<code>class</code>	definir un modele de donnee
<code>\$this</code>	soi-meme, dans une classe
<code>namespace</code>	organiser les classes
<code>container / row / col</code>	grille Bootstrap
<code>card</code>	bloc visuel Bootstrap
<code>form-control</code>	champ de saisie stylise

6.2 Quel outil utiliser

- Je veux recuperer un identifiant dans l'URL -> `$_GET`
- Je veux recuperer une valeur d'un formulaire -> `$_POST`
- Je veux garder une donnee d'une page a l'autre -> `$_SESSION`
- Je veux envoyer l'utilisateur vers une autre page -> `header('Location: ...')` suivi de `exit`
- Je veux afficher plusieurs cartes en grille -> `row row-cols-... g-4` avec des `col`
- Je veux un champ de formulaire bien stylise -> `form-control`
- Je veux regrouper donnees et actions -> une classe (`class`)

7 Regles d'évolution de ce document

- Ce document suit les memes regles que la base de reference Dart/Flutter : pas de retour a zero, mise a jour progressive, statut honnete de chaque notion.
- Quand une nouvelle page ou une nouvelle classe est ajoutée au projet web, les notions correspondantes sont ajoutées ici.
- Quand le projet passera a une vraie base de données, une nouvelle section sera ajoutée pour PDO ou l'outil choisi, sans supprimer ce qui est déjà documenté sur les sessions.